

Estudo Dirigido

Comparing LLM-Based Policy Generation and Reinforcement Learning for Embodied Control

Jarne Demoen

Vrije Universiteit Brussel / Universidade Federal de Santa Catarina

29/03/2026

1 Introduction

Reinforcement Learning (RL) has become one of the most widely used paradigms for training agents that interact with dynamic environments. In RL, an agent learns a policy by interacting with the environment and optimizing a reward signal through trial and error. Popular deep reinforcement learning algorithms such as Proximal Policy Optimization (PPO), Soft Actor-Critic (SAC), and Deep Deterministic Policy Gradient (DDPG) have demonstrated strong performance in continuous control tasks and robotics. However, reinforcement learning typically requires a large number of interactions with the environment before a useful policy emerges. This is because learning usually starts from a random policy and gradually improves through stochastic exploration. As a result, RL can be computationally expensive and sample inefficient, especially for complex environments. Recent research has explored the use of Large Language Models (LLMs) as controllers for embodied agents (Carvalho and Nolfi 2026). Carvalho and Nolfi propose a method in which LLMs generate control strategies directly from textual descriptions of the agent, environment, and task objective. The generated policy maps continuous observation vectors to continuous action vectors and can therefore directly control the agent’s actuators.

In this approach, the LLM first generates an initial control strategy using its prior knowledge and reasoning capabilities. The strategy is then iteratively refined using feedback obtained from the agent’s interaction with the environment. During each iteration, the model receives sensory-motor data and reward information from the previous evaluation and proposes improvements to the controller. This iterative policy refinement framework integrates symbolic reasoning from the LLM with sub-symbolic sensory-motor data collected during interaction. Unlike many existing robotics approaches, it does not require demonstration datasets or model fine-tuning.

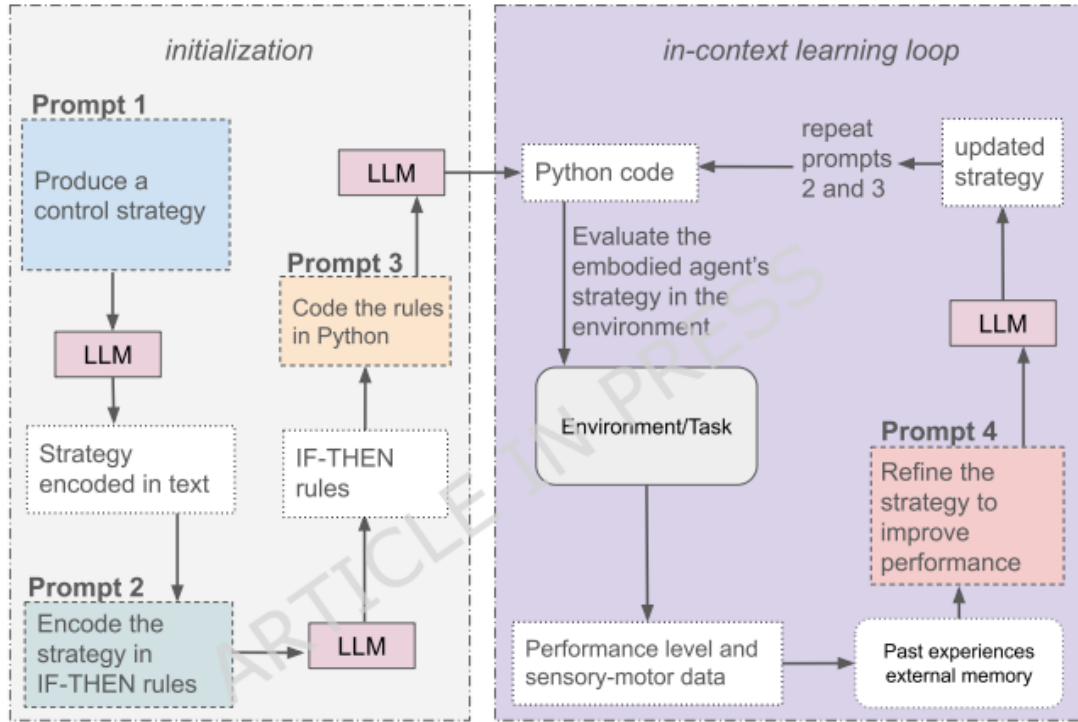


Figure 1: Overview of the prompting method enabling LLMs to directly control the actuators of embodied agents.

While this method has shown promising results, it remains unclear how it compares to traditional reinforcement learning approaches. Furthermore, recent frameworks such as the ART (Agent Reinforcement Training) system introduce automated reinforcement learning mechanisms that can train LLM-based agents using reward signals derived from task execution (*OpenPipe/ART at Auto-Rl 2026*).

2 Research Objectives

The objective of this project is to investigate whether LLM-based policy generation combined with iterative refinement can serve as a viable alternative to classical reinforcement learning methods for embodied control tasks. In particular, the project will explore whether reinforcement learning techniques applied to LLM-based agents using the ART framework can improve the performance of LLM-generated controllers. The main research questions are:

- Can LLM-generated control policies achieve performance comparable to standard (deep) reinforcement learning algorithms?
- Does applying reinforcement learning to the LLM itself using the ART framework improve the quality of generated policies?
- How do the different approaches compare in terms of learning speed, sample efficiency, and policy robustness?

3 Methodology

This study will compare three approaches to learning control policies for embodied agents.

3.1 Standard Reinforcement Learning Baselines

Several commonly used deep reinforcement learning algorithms will be implemented as baselines. These algorithms learn policies directly from interaction data using gradient-based optimization. The algorithms considered in this project include:

- Proximal Policy Optimization (PPO)
- Soft Actor-Critic (SAC)
- Deep Deterministic Policy Gradient (DDPG)

These algorithms will be trained on standard continuous control environments and serve as reference methods for comparison.

3.2 LLM-Based Iterative Policy Refinement

The second approach follows the methodology proposed by Carvalho and Nolfi (2026). In this framework, an LLM generates an initial control strategy based on a textual description of the agent, environment, and task objective. The generated strategy is converted into executable code representing the controller. The controller is then evaluated in the environment, producing sensory-motor data and reward signals. During each iteration of the learning loop, the LLM receives:

- The current control strategy
- The best strategy discovered so far
- Performance scores from previous evaluations
- Sensory-motor data collected during the most recent episode

Using this information, the LLM proposes modifications to the policy in order to improve performance. This process is repeated for multiple iterations until satisfactory performance is achieved.

3.3 LLM-Based Control with ART Reinforcement Training

The third approach extends the previous method by incorporating reinforcement learning using the ART (Agent Reinforcement Training) framework. ART allows LLM-based agents to be trained using reward signals derived from task performance. Instead of relying solely on iterative prompting, the model can be further improved by updating its parameters based on rewarded trajectories generated during interaction. In this setup, the LLM will generate policies and interact with the environment as before, but the ART framework will collect trajectories and use them to train the model through reinforcement learning updates. This allows the LLM to adapt its behavior based on accumulated experience rather than relying exclusively on prompt-based reasoning.

4 Experimental Setup

Experiments will be conducted using simulation environments from the Gymnasium library. These environments provide standardized benchmarks for evaluating control policies. Candidate environments include:

- CartPole
- Pendulum
- MountainCarContinuous
- Acrobot
- Ant

Each approach will be evaluated using identical environments and comparable computational resources. Performance will be evaluated using the following metrics:

- Average cumulative reward
- Learning speed (episodes required to reach a performance threshold)
- Policy robustness across different initial conditions
- Sample efficiency (number of interactions required for learning)

Multiple experimental runs with different random seeds will be performed to ensure statistical reliability. /

5 Work Plan and Timeline

The work plan starts on Monday 1 June 2026 and ends on Friday 10 July 2026. The timeline is designed to produce a complete research artifact even if the ART component requires adjustment.

Period	Focus	Main activities	Expected output
1–5 June	Setup and research framing	Finalize scope, define environments, create repository structure, review the Carvalho and Nolfi method, inspect ART documentation, set up Gymnasium/RL dependencies, define logging format.	Research protocol, repository skeleton, environment sanity checks.

Period	Focus	Main activities	Expected output
8–12 June	Classical baselines	RL Implement or configure PPO, SAC, and DDPG baselines; run initial experiments on simple environments; define reward thresholds; verify reproducibility with fixed seeds.	Baseline training scripts, initial baseline results, experiment configuration files.
15–19 June	LLM policy generation loop	Design structured prompts, implement controller code generation interface, execute one-shot generated policies, build safety checks for generated Python code, store generated policies and evaluation logs.	Working one-shot LLM controller pipeline with logs and first reward results.
22–26 June	Iterative policy refinement	Implement refinement loop using performance feedback and sensory-motor traces; compare one-shot and refined controllers; tune feedback format; analyze failure cases.	Iterative refinement results, policy history, preliminary comparison against RL baselines.
29 June–3 July	ART feasibility and extension	Attempt ART integration; formulate controller generation as a reward-based training task; run a minimal prototype if feasible; otherwise document blockers and create a technical integration design.	ART prototype or feasibility report, updated experimental comparison.
6–10 July	Final analysis and report	Consolidate results, create plots and tables, interpret findings, write final report, document limitations and future work, clean repository for reproducibility.	Final report, final code repository, figures, tables, and presentation-ready summary.

6 Detailed Weekly Milestones

Week 1: Setup, Scope, and Experimental Protocol

The first week focuses on reducing ambiguity. The project should not begin with large-scale experimentation before the environments, metrics, and logging format are fixed. The main tasks are to install the necessary packages, test selected Gymnasium environments, define a standard evaluation function, and create a consistent file structure for experiment outputs.

Deliverable: A short protocol document describing the selected environments, reward thresholds, number of seeds, evaluation episodes, logging format, and computational constraints.

Week 2: Reinforcement Learning Baselines

The second week establishes reference results. Classical RL baselines should be trained under reasonable budgets and evaluated using the same evaluation function that will later be used for LLM-generated policies. The baseline results do not need to be state-of-the-art, but they must be consistent, reproducible, and well documented.

Deliverable: Baseline results for at least two environments, including reward curves, final evaluation rewards, and configuration files.

Week 3: One-Shot LLM Controller Generation

The third week implements the first LLM-based method. The goal is to obtain executable controllers generated from structured prompts. Since generated code can be unstable, the pipeline should include syntax checks, runtime checks, bounded actions, exception handling, and logging of failed attempts. One-shot performance should be measured before refinement is introduced.

Deliverable: A working one-shot LLM controller generation pipeline and an initial comparison against RL baselines.

Week 4: Iterative Policy Refinement

The fourth week introduces feedback-driven improvement. The LLM should receive structured information about previous performance, including reward values, selected trajectory summaries, and the best controller so far. The key question is whether repeated refinement improves controller quality compared to one-shot generation.

Deliverable: Iterative refinement experiments showing whether performance improves across refinement iterations.

Week 5: ART Integration or Feasibility Analysis

The fifth week is dedicated to the ART component. The ideal outcome is a minimal ART-based training prototype in which generated policy outputs are evaluated through environment rewards. However, if integration is not feasible within the available time, this week should produce a rigorous feasibility analysis: required interfaces, reward definition, generated data format, compute requirements, and technical blockers.

Deliverable: Either a minimal ART prototype or a detailed ART integration design with clear reasons for any limitations.

Week 6: Analysis, Writing, and Finalization

The final week focuses on producing a research-worthy final output. The report should not merely describe implementation steps; it should interpret the results in relation to the research questions. It should explain when LLM-generated policies are useful, where they fail, whether refinement helps, and whether ART appears promising for future work.

Deliverable: Final research report, cleaned code repository, experiment logs, figures, tables, and conclusions.

7 Reproducibility Plan

To make the project research-worthy, all experiments should be reproducible. The repository should contain:

- a `README.md` explaining how to install dependencies and run experiments;
- fixed random seeds for repeated experiments;
- configuration files for each environment and method;
- raw logs of rewards, episode lengths, generated policies, and refinement iterations;
- scripts for plotting reward curves and summary tables;
- a clear separation between baseline RL code, LLM policy generation code, and ART-related experiments.

Generated LLM controllers should be stored with metadata, including model name, prompt version, environment, timestamp, iteration number, and evaluation reward. This is important because prompt-based methods can vary substantially across runs.

8 Risk Management and Fallback Strategy

9 Expected Final Report Structure

The final report should follow a research-oriented structure:

1. **Introduction:** motivation, problem statement, and contribution.
2. **Background:** classical RL, LLM-based control, iterative policy refinement, and ART.
3. **Research Questions:** formal statement of the questions and hypotheses.
4. **Methodology:** environments, algorithms, prompting procedure, refinement loop, ART setup, metrics, and reproducibility details.
5. **Results:** baseline results, one-shot LLM results, refinement results, and ART feasibility or performance results.
6. **Discussion:** interpretation, strengths, weaknesses, threats to validity, and relation to the research questions.
7. **Conclusion:** main findings and future research directions.

Risk	Impact	Mitigation
ART integration is too time-consuming	The third approach may not produce complete experimental results	Treat ART as a feasibility extension and document a minimal integration design
LLM-generated code is invalid or unsafe	Experiments may fail or produce unusable controllers	Add syntax checks, action clipping, execution timeouts, and controlled evaluation wrappers
RL baselines require too much tuning	Baseline comparison may become unfair or incomplete	Use established implementations and fixed training budgets instead of excessive tuning
Complex environments such as Ant are too expensive	The study may become too broad for the available period	Prioritize simpler environments and include Ant only as an optional stretch goal
High variability across seeds	Results may be difficult to interpret	Report mean and variability, and discuss robustness explicitly

Table 2: Main project risks and mitigation strategies.

10 Expected Contributions

The expected contributions of this project are:

- An implementation of LLM-based policy generation for embodied control tasks.
- A comparative evaluation between traditional reinforcement learning methods and LLM-based controllers.
- An investigation of whether reinforcement learning applied to LLM agents using ART improves controller performance.
- Insights into the advantages and limitations of reasoning-based policy generation compared to gradient-based learning.

11 Conclusion

This project explores a novel paradigm for training autonomous agents by combining the reasoning capabilities of large language models with reinforcement learning techniques. By comparing classical reinforcement learning algorithms with LLM-generated policies and ART-enhanced LLM agents, the study aims to evaluate whether LLM-based approaches can provide faster policy initialization and improved learning efficiency in embodied control tasks.

References

Carvalho, Jonata Tyska and Stefano Nolfi (Mar. 2026). “Sensory-Motor Control with Large Language Models via Iterative Policy Refinement”. In: *Scientific Reports*. ISSN: 2045-2322. DOI: 10.1038/s41598-026-42091-0. (Visited on 03/29/2026).
OpenPipe/ART at Auto-Rl (2026). <https://github.com/OpenPipe/ART/tree/auto-rl>. (Visited on 03/29/2026).

Supervisor Approval

I, Prof. Jônata Tyska Carvalho, approve this proposal for the *Estudo Dirigido* course and agree to supervise the proposed project.

Supervisor: Prof. Jônata Tyska Carvalho

Signature:_____

Date:_____